

11. uzd

Jekaterīna Butkeviča

2025. gads 24. decembris

Uzdevums:

1. Atrodiet internetā kādu interesantu datu kopu dimensiju redukcijas uzdevumam (ja nekas cits neatrodas, izmantojiet WEKAs faila unbalanced datus).
2. Mēģiniet atrisināt šo uzdevumu, izmantojot kādu no jums pieejamām TriMap vai PaCMAP (vai abām) realizācijām:
 - Iekrāsojiet dažādu klašu objektus atšķirīgās krāsās. Mainiet galvenos parametrus ("xxx kaimiņu skaits") no minimālās vērtības līdz ir iegūts stabils rezultāts.
 - Uzrakstiet atskaiti par saviem piedzīvojumiem, ietverot izmantotos programmu pirmkodus un iegūto rezultātu attēlus.
 - Ko varat secināt par rezultātu atkarību no galvenā parametra? Vai izdevās pārspēt t-SNE un/vai UMAP rezultātus?

Dati

Tā kā iepriekšējos uzdevumu izvēlējos veikt ar unbalanced datiem, turpināšu izmantot tieši tos, lai vieglāk uztvertu metožu atšķirības.

Atgādinājumam — WEKA fails unbalanced ir viens no WEKA Artificial Datasets kolekcijas datiem, kas veidoti, lai simulētu nevienmērīgi sadalītas klases. Dati satur sintētiskus atribūtus, kas ģenerēti, izmantojot Weka Balanced Network ar Gaussian cluster principu.

Ielasu datu kopu, klases saglabāju atsevišķi, un atsevišķā mainīgajā pārnesu tikai skaitliskos atribūtus.

```
library(foreign)
unbalanced <- read.arff("unbalanced.arff")
unbalanced_labels <- unbalanced$Outcome
unbalanced_numeric <- unbalanced[apply(unbalanced, is.numeric)]
```

Tehniskie izaicinājumi

PaCMAP un TriMap oriģināli ir izstrādāti Python valodā un nav tieši pieejami R. Risinājums bija izmantot pakotni reticulate, kas ļauj R vidē palaist Python bibliotēkas. Tā kā man nav pieredzes ar Python, lai izmantotu šīs bibliotēkas R vidē, izmantoju Gemini piedāvātos risinājumus.

```
library(reticulate)
```

```
# Vide
py_install("annoy", envname = "r-dimension-reduction", pip = TRUE)

## Using virtual environment "r-dimension-reduction" ...

py_install(c("pacmap", "trimap"), envname = "r-dimension-reduction", pip =
TRUE)

## Using virtual environment "r-dimension-reduction" ...
```

Sākotnēji radās sarežģījumi ar annoy bibliotēkas kompilāciju. Problēma tika atrisināta, instalējot Microsoft C++ Build Tools un izveidojot specifisku virtuālo vidi r-dimension-reduction, kas nodrošināja veiksmīgu pacmap un trimap uzstādīšanu.

```
library(reticulate)
library(ggplot2)
library(patchwork)
library(foreign)
```

Izveidoju vienotu funkciju rezultātu attēlošanai, lai turpmāk neaizņemtu lieku vietu.

```
plot_res <- function(res, title) {
  df <- data.frame(X1 = res[,1], X2 = res[,2], Outcome = unbalanced_labels)
  ggplot(df, aes(X1, X2, color = Outcome)) +
    geom_point(alpha = 0.5) +
    theme_minimal() +
    ggtitle(title)
}
```

Pārbaudu, vai tiek izmantota vajadzīgā vide.

```
use_virtualenv("r-dimension-reduction", required = TRUE)
```

PaCMAP

PaCMAP galvenais parametrs ir `n_neighbors`, kas nosaka, cik daudz tuvāko kaimiņu punkti tiek ņemti vērā, veidojot grafu. Eksperimentēšu ar trim vērtībām: 5, 20 un 50.

Importēju Python pakotnes, sagatavoju datus un palaidu PaCMAP ar trim dažādām `n_neighbors` vērtībām. Rezultātus saglabāju atsevišķos objektos.

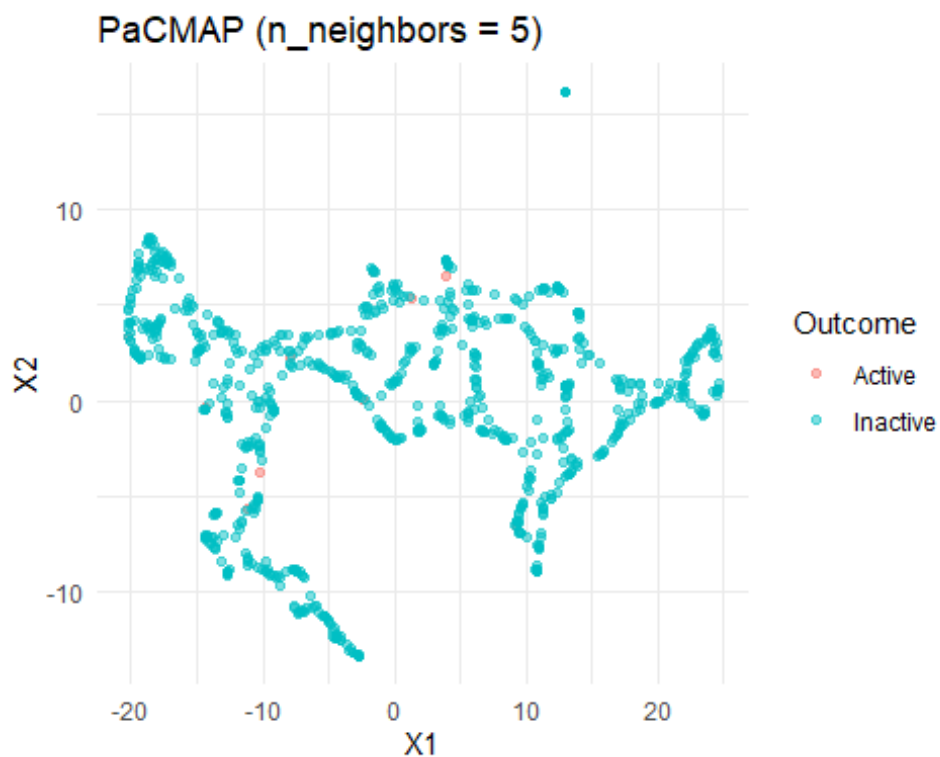
```
import pacmap
import numpy as np

X = np.array(r.unbalanced_numeric)

# Eksperimentēju ar 3 vērtībām
pm_res1 = pacmap.PaCMAP(n_neighbors=5).fit_transform(X) # Minimāla vērtība
pm_res2 = pacmap.PaCMAP(n_neighbors=20).fit_transform(X) # Vidēja vērtība
pm_res3 = pacmap.PaCMAP(n_neighbors=50).fit_transform(X) # Stabils rezultāts
```

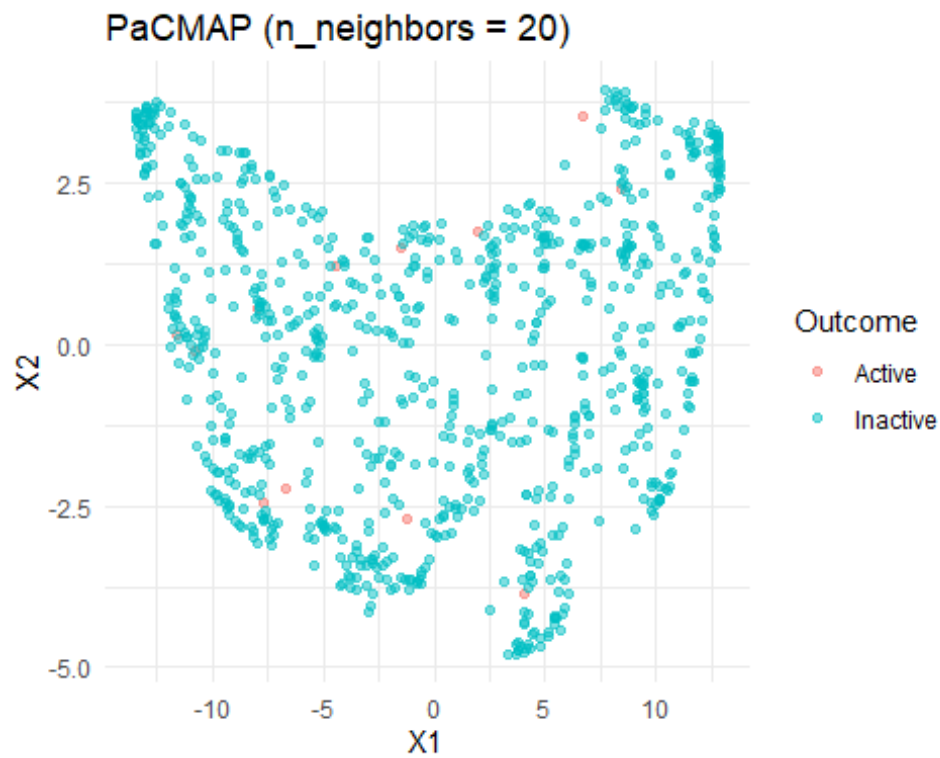
Vizualizēju rezultātus.

```
plot_res(py$pm_res1, "PaCMAP (n_neighbors = 5)")
```



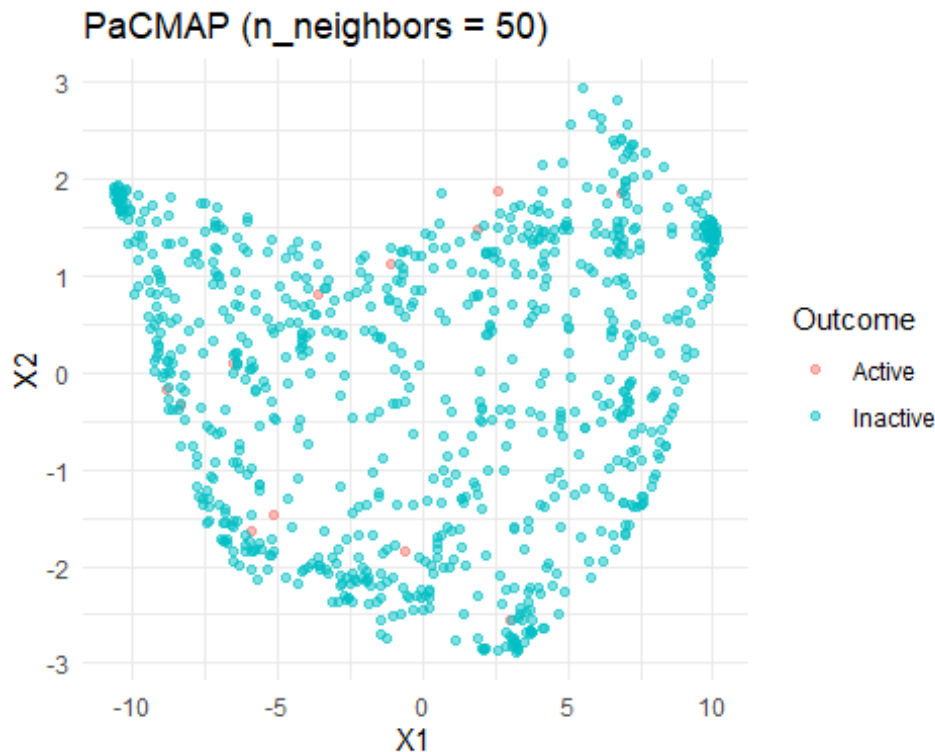
Ar `n_neighbors` parametra vērtību 5 iegūtā attēlā punkti veido līnijveida struktūras. Nodalītu klasteru nav, un grupas nav atdalītas.

```
plot_res(py$pm_res2, "PaCMAP (n_neighbors = 20)")
```



Ar `n_neighbors` parametra vērtību 20 iegūtais rezultāts izteikti atšķiras no iepriekšējā variantā. Punkti ir vairāk izklaidēti, veidojot mākonī, kas nosmailojas tā galos. Novērojamas nelielas grupēšanās zonas (blīvākas vietas), taču noformētu klasteru nav. Grupas nav atdalītas.

```
plot_res(py$pm_res3, "PaCMAP (n_neighbors = 50)")
```



Ar `n_neighbors` parametra vērtību 50 iegūtais rezultāts maz atšķiras no iepriekšējā attēla. Blīvākas zonas joprojām ir novērojamas, taču tās ir mazāk izteiktas. Mākoņa centrā punktu blīvums ir samazinājies. Punkti mākoņa galos ir sakoncentrējušies divos klasteros. Izteikti nodalītu klasteru nav. Grupas nav atdalītas.

TriMap

TriMap izmanto tripletus, lai definētu datu struktūru. Algoritmā ir pieejami vairāki parametri: `n_outliers` – izlecēju skaits tripletu veidošanai, `n_random` – nejaušo tripletu skaits uz punktu, kā arī galvenais parametrs `n_inliers` – tuvāko kaimiņu skaits, kas tiek izmantots tripletu veidošanai. Lai izvairītos no pārmērīgi liela parametru kombināciju skaita, eksperimentos tiks mainīts tikai pēdējais no minētajiem parametriem.

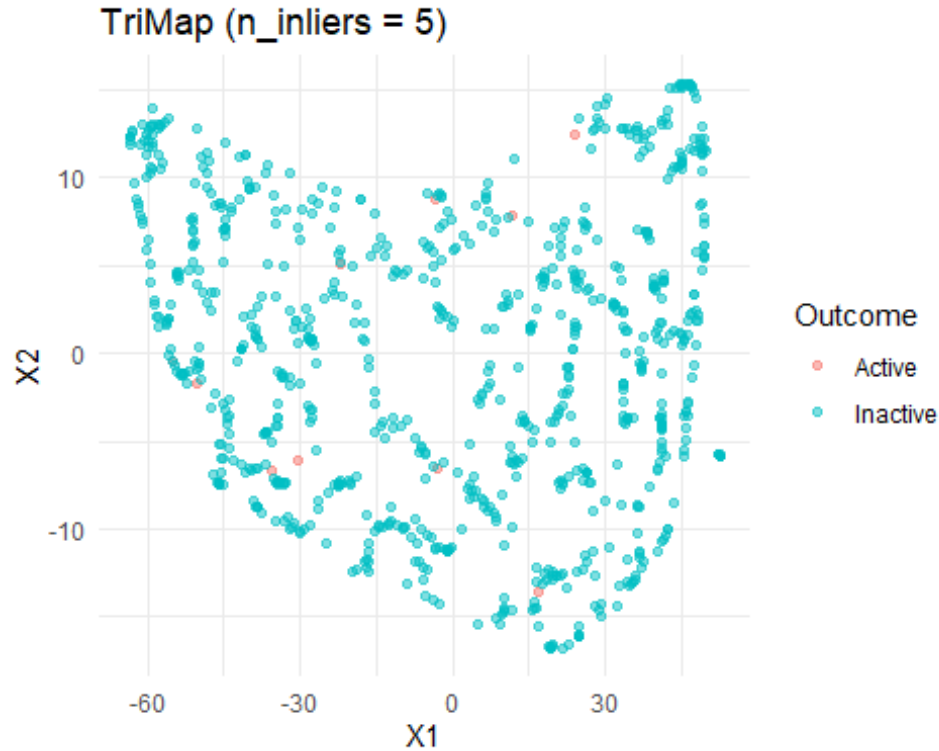
Tika importēta attiecīgā pakotne un veikti eksperimenti ar trim dažādām `n_inliers` vērtībām. Iegūtie rezultāti tika saglabāti atsevišķos mainīgajos.

```
import trimap

# Eksperimentēju ar 3 vērtībām
tm_res1 = trimap.TRIMAP(n_inliers=5).fit_transform(X)
tm_res2 = trimap.TRIMAP(n_inliers=20).fit_transform(X)
tm_res3 = trimap.TRIMAP(n_inliers=50).fit_transform(X)
```

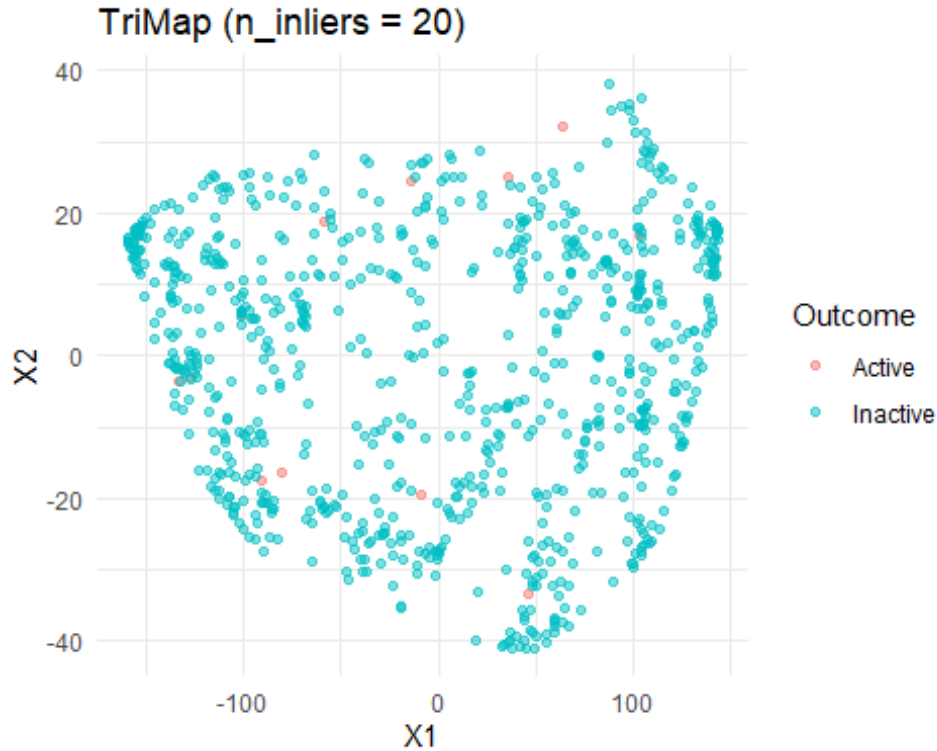
Vizualizēju rezultātus.

```
plot_res(py$tm_res1, "TriMap (n_inliers = 5)")
```



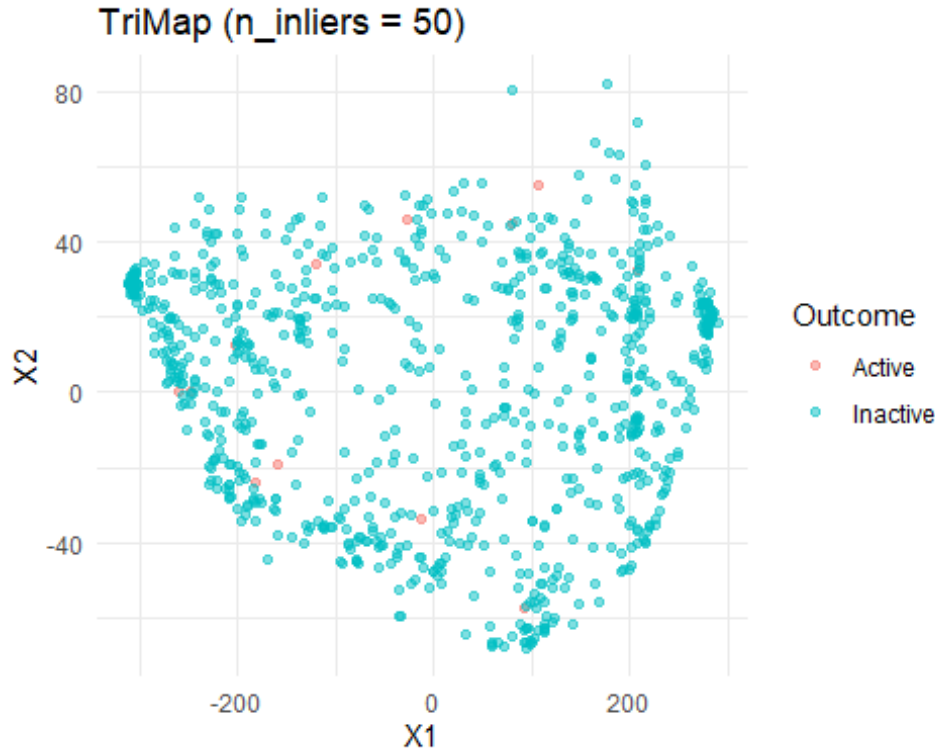
Izmantojot `n_inliers` parametra vērtību 5, iegūtajā attēlā punkti, līdzīgi kā pirmajā PaCMAP variantā, veido diegveida struktūras, tomēr tās nav tik izteiktas kā PaCMAP gadījumā. Datu mākonim jau ir raksturīga forma, savukārt diegveida struktūrās punkti grupējas tās iekšienē — globālā datu struktūra ir atveidota jau no paša sākuma. Klases nav atdalītas.

```
plot_res(py$tm_res2, "TriMap (n_inliers = 20)")
```



Ar parametra `n_inliers` vērtību 20 izveidotais attēls pēc mākoņa globālās struktūras būtiski neatšķiras no iepriekšējā variantā, tomēr ir mainījusies mākoņa lokālā struktūra — diegveida grupējumi vairs netiek novēroti, un punkti ir izklaidēti ievērojami vienmērīgāk. Vienlaikus punktu blīvums dažādās mākoņa daļās joprojām atšķiras, un mākoņa galos punktiem ir lielāka koncentrācija. Klases nav skaidri atdalītas.

```
plot_res(py$tm_res3, "TriMap (n_inliers = 50)")
```



Ar parametra `n_inliers` vērtību 50 iegūtais attēls maz atšķiras no iepriekšējā variantā. Punktu blīvums mākoņa centrā kļūva mazāks, savukārt mākoņa perifērijā — lielāks. Mākoņa galos punktu blīvums arī palielinājās.

Secinājumi:

1. Tas, ka nevienā no mēģinājumiem klases netika skaidri atdalītas, apstiprina manis iepriekšējā uzdevumā izvirzīto hipotēzi, ka klase `Active` satur punktus, kas faktiski ir daļa no lielākās `Inactive` klases un nav unikāli pēc savām īpašībām.
2. `TriMap` algoritms jau ar pirmo mēģinājumu pietiekami precīzi atveido datu globālo struktūru, savukārt `PaCMAP` algoritmam ar zemu `n_neighbors` vērtību to izdarīt neizdevās.
3. Izteikti atšķirīgais rezultāts, kas parādījās, izmantojot `PaCMAP` ar `n_neighbors = 5`, visticamāk radās tādēļ, ka šajā gadījumā algoritms saņem ļoti maz informācijas par punktu savstarpējo saistību kopējā datu mākonī un ņem vērā tikai šauras, lokālas pārejas no viena punkta uz nākamo.
4. Abi algoritmi beigās, nonākot pie stabila rezultāta, piedāvā ļoti līdzīgu ainu, jo pie pietiekama informācijas daudzuma tie sāk atklāt vienu un to pašu – datu globālo struktūru.

Salīdzinājums ar t-SNE un UMAP

Iegūtie rezultāti daļēji atgādina rezultātus, kas tika iegūti ar t-SNE algoritmu pie perplexity = 30 vai ar UMAP algoritmu pie n_neighbors = 45. Līdzība iegūtajos rezultātos, iespējams, liecina par to, ka ir sasniegta datu kopas “patiesā forma”. Visi četri analizētie algoritmi ir nelineāras dimensiju redukcijas metodes. t-SNE ar perplexity = 30 un UMAP ar n_neighbors = 45 ir iestatījumi, kas liek algoritmiem ņemt vērā lielāku kaimiņu skaitu, tādējādi samazinot fokusu uz lokālām detaļām un pastiprinot datu kopas globālās struktūras attēlojumu.

Analizētā unbalanced datu kopa pati par sevi lielā mērā nosaka iegūto rezultātu — jebkurš pietiekami avancēts algoritms, kas cenšas saglabāt datu struktūru, gala rezultātā atveidos vairāk vai mazāk līdzīgu telpisko izvietojumu.

Tomēr PaCMAP un TriMap algoritmi stabili datu globālās struktūras attēlojumu sasniedza jau pie mazākām galveno parametru vērtībām, īpaši izteikti tas novērojams TriMap gadījumā.